FinMarket-Enterprise

Phase II — Database and Security Integration

Member 4: @Bruno-Oliveira2 — Email and Facebook Delivery

Branch: `feature/delivery-p2-m4`
Assigned module: `backend/src/modules/delivery/`
Primary ownership: Delivery database persistence, email and Facebook delivery integration, delivery queue management, rate limiting, delivery tracking, recipient eligibility, and delivery authorization.

---

1. Project Overview

FinMarket-Enterprise is a financial advisory marketing and client acquisition platform designed to help financial advisors manage prospective clients, organize marketing campaigns, deliver authorized marketing communications, automate follow-up activities, and monitor marketing performance.

The application consists of five main modules:

1. Identity and Advisor Management
2. Lead Management (CRM)
3. Marketing Campaigns and Templates
4. Email and Facebook Delivery
5. Automation and Analytics

During Phase I, the Delivery module was implemented using temporary in-memory storage. Core API endpoints were established for creating delivery jobs, retrieving delivery history, and processing queued deliveries through a mock provider.

Initial delivery models, supported communication channels, recipient consent checks, campaign eligibility validation, and delivery status handling were also introduced.

Phase II focuses on replacing temporary delivery storage with MongoDB, implementing secure and persistent delivery processing, introducing controlled Gmail SMTP

integration, supporting authorized Facebook Page publishing where permitted, and enforcing role-based access.

The existing Delivery architecture and API contracts should be preserved wherever practical.

1.1 Phase II Objectives

The following responsibilities must be completed:

- Replace in-memory DeliveryJob storage with MongoDB using Mongoose.
- Implement persistent delivery jobs and status tracking.
- Maintain relationships between DeliveryJobs, Campaigns, Leads, and AdvisorProfiles.
- Protect delivery endpoints using JWT authentication.
- Enforce role-based authorization and resource ownership.
- Allow Marketing Specialists to perform only permitted delivery operations for assigned Campaigns.
- Implement controlled email delivery using Gmail SMTP.
- Support authorized Facebook Page publishing through the Meta Graph API where access and permissions are available.
- Retain the existing mock delivery provider for independent testing and demonstrations.
- Implement delivery queue processing and rate limiting.
- Prevent duplicate or unauthorized delivery execution.
- Validate recipient marketing consent and unsubscribe status.
- Implement delivery scheduling and processing restrictions.
- Record successful and failed delivery outcomes.
- Securely manage external provider credentials.
- Preserve existing Phase I API functionality.
- Implement Postman and automated tests for persistence, authorization, validation, and delivery processing.

The Delivery module must be independently developable and testable without requiring completion of the other feature modules.

---

2. Development Environment and Team Independence

The Dockerized backend and MongoDB infrastructure will be prepared by Member 1 and merged into `main` before Phase II feature development begins.

Once the infrastructure is available, the repository must be cloned or updated from the latest `main` branch.

Development must occur in:

`feature/delivery-p2-m4`

The shared Docker configuration, MongoDB connection, JWT authentication middleware, role constants, and error-handling contracts must be reused.

Member 4 must not independently replace or modify shared infrastructure.

The Delivery module must remain responsible for its own:

- Mongoose models.
- MongoDB repositories.
- Delivery services and business rules.
- Controllers and routes.
- Delivery queue management.
- Provider integrations.
- Rate limiting.
- Delivery status tracking.
- Recipient eligibility checks.
- Module-specific authorization.
- Unit and integration tests.
- Module documentation.

Changes to shared files such as `backend/src/app.js`, `backend/src/routes/index.js`, `backend/src/config/`, or shared authentication middleware must be coordinated with Member 1.

2.1 Independent Development Requirements

The following approach must be followed:

1. Existing Delivery API contracts must be preserved wherever practical.
2. The in-memory delivery repository must be replaced without requiring changes to other modules.
3. Stable MongoDB ObjectIds must be used to represent relationships.
4. Sample Advisors, Campaigns, Leads, and accepted specialist assignments may be created through test fixtures.
5. Shared authentication and relationship lookup contracts may be mocked during isolated tests.
6. Campaign and Lead lookup contracts must be used without modifying their modules.
7. Delivery providers must use interchangeable interfaces so the mock provider can be used without Gmail or Meta credentials.
8. The Delivery module must not require the Automation module to operate during independent development.
9. Scheduled deliveries must be testable without requiring another member's automation scheduler.
10. The module must remain functional when external email or Facebook services are unavailable.

A mock provider must remain available for local development and testing.

Real provider integrations must not be required for starting the backend or running the automated test suite.

---

3. Shared User Roles and Permissions

The application supports three roles:

3.1 Administrator — `ADMIN`

Administrators manage platform-level user accounts, permissions, and administrative operations.

Administrative access to Delivery operations must follow explicitly defined permissions.

The Administrator role must not automatically bypass recipient consent, ownership, or other delivery safety rules.

3.2 Financial Advisor — `ADVISOR`

Financial Advisors own their Campaigns and associated marketing deliveries.

An authenticated Advisor must be allowed to:

- Queue deliveries for their own eligible Campaigns.
- Retrieve delivery history belonging to their own Campaigns.
- Process eligible delivery jobs.
- Review successful and failed delivery results.
- Schedule permitted deliveries.
- Access delivery status information.
- Use authorized communication channels.

An Advisor must not access or process another Advisor's delivery jobs without explicitly authorized access.

3.3 Marketing Specialist — `MARKETING_SPECIALIST`

Marketing Specialists may perform authorized delivery operations on behalf of Financial Advisors.

A Marketing Specialist may:

- Queue deliveries for Campaigns explicitly assigned to them.
- Process permitted delivery jobs for assigned Campaigns.
- View delivery history for assigned Campaigns.
- Review delivery results and errors.
- Prepare authorized email or Facebook communications.

A Marketing Specialist must not:

- Queue deliveries for unrelated Campaigns.
- Process delivery jobs belonging to unassigned Campaigns.
- Access unrestricted Lead contact information.
- Change Campaign ownership.
- Modify AdvisorProfile information.
- Manage advisor-specialist invitations.
- Assign themselves to Campaigns.
- Access provider credentials.
- Bypass marketing consent or unsubscribe requirements.

A Marketing Specialist must satisfy both conditions before performing delivery operations:

1. An accepted relationship exists with the Campaign's owning Advisor.
2. The Marketing Specialist is assigned to the relevant Campaign.

The backend must verify both conditions.

---

4. Database Identifiers and Relationships

The following identifier conventions must be used consistently.

4.1 Identifier Definitions

Field	Meaning	Referenced Collection
`userId`	Unique identifier of a registered user	`users`
`advisorId`	Unique identifier of an AdvisorProfile	`advisorProfiles`
`specialistUserId`	User ID belonging to a Marketing Specialist	`users`
`campaignId`	Unique identifier of a Campaign	`campaigns`
`templateId`	Unique identifier of a Template	`templates`
`leadId`	Unique identifier of a Lead	`leads`
`deliveryJobId`	Unique identifier of a DeliveryJob	`deliveryJobs`
`ownerId`	User ID of an owner where defined	`users`
`eventId`	Unique application-level identifier of an event	`activityEvents`

Important distinction:

`advisorId` always references `AdvisorProfile._id`, not `User._id`.

For example:

- `User._id = A`
- `AdvisorProfile._id = B`
- `AdvisorProfile.userId = A`
- `Campaign.advisorId = B`
- `DeliveryJob.advisorId = B`

For a Marketing Specialist:

- `User._id = C`
- `Campaign.assignedSpecialistIds = [C]`

Marketing Specialists do not have AdvisorProfile records.

Their delivery permissions must be determined using their authenticated User ID and assigned Campaign relationships.

4.2 Required Relationships

AdvisorProfile → DeliveryJob

Relationship: One-to-many.

An AdvisorProfile may own multiple DeliveryJobs.

`DeliveryJob.advisorId → AdvisorProfile._id`

The DeliveryJob must belong to the same Advisor as its Campaign.

Campaign → DeliveryJob

Relationship: One-to-many.

A Campaign may produce multiple DeliveryJobs.

Each DeliveryJob must reference an existing Campaign.

`DeliveryJob.campaignId → Campaign._id`

The Campaign's status, channel, and ownership must be validated before delivery processing.

Lead → DeliveryJob

Relationship: One-to-many.

A Lead may receive multiple authorized email communications.

`DeliveryJob.leadId → Lead._id`

For email deliveries targeting CRM leads, a valid `leadId` must be required.

The Lead must belong to the Campaign's owning Advisor.

The Lead must also satisfy marketing consent and audience eligibility requirements.

For Facebook Page publishing, a Lead reference may be unnecessary because the content is published to a Page rather than sent directly to an individual Lead.

Campaign → Marketing Specialist

Relationship: Many-to-many.

Campaigns maintain specialist assignments using:

`Campaign.assignedSpecialistIds → User._id[]`

Member 4 must use the Campaign authorization contract to verify specialist access.

Member 4 must not create a separate specialist assignment system.

DeliveryJob → ActivityEvent

Delivery processing may produce business events such as successful or failed delivery outcomes.

These events must follow the agreed shared event contract.

ActivityEvent persistence and automation processing belong to Member 5.

---

5. Member 4 — Assigned Phase II Responsibilities

5.1 DeliveryJob Mongoose Model and Persistence

Owned directory: `backend/src/modules/delivery/`

The existing DeliveryJob model must be reviewed and updated for persistent MongoDB storage.

Collection:

`deliveryJobs`

Recommended fields include:

- `_id`
- `advisorId`
- `campaignId`
- `leadId`
- `channel`
- `recipientEmail`
- `content`
- `deliveryStatus`
- `scheduledAt`
- `sentAt`
- `providerMessageId`
- `errorMessage`
- `createdAt`
- `updatedAt`

Additional processing metadata may be introduced where necessary, such as:

- `processingStartedAt`
- `attemptCount`
- `idempotencyKey`
- `provider`

The existing Phase I schema fields and allowed values must be preserved wherever practical.

Requirements include:

1. Required field validation.
2. Correct Mongoose field types.
3. Supported communication channels.
4. Supported delivery statuses.
5. Appropriate default values.
6. Timestamp management.
7. AdvisorProfile references.
8. Campaign references.
9. Optional Lead references depending on the delivery channel.
10. Relevant indexes for status, campaign, advisor, and scheduling queries.

The DeliveryJob model must use `AdvisorProfile._id` for `advisorId`.

The existence and validity of referenced Campaigns and Leads must be checked through the appropriate service contracts.

Mongoose ObjectId validation alone is not sufficient to establish reference existence.

5.2 Replace the In-Memory Repository

Owned repository directory: `backend/src/modules/delivery/repositories/`

The existing temporary delivery storage must be replaced with Mongoose database queries.

Required repository functionality includes:

- Create DeliveryJob.
- Retrieve DeliveryJob by ID.
- List DeliveryJobs.
- Filter delivery history.
- Find pending deliveries.
- Find scheduled deliveries that are due.
- Claim a pending delivery for processing.
- Update delivery status.
- Record provider responses.
- Record delivery failures.
- Count delivery results where required.

All production delivery records must be stored in MongoDB.

Database queries must remain in the repository layer.

Controllers must not directly query MongoDB.

The delivery service must remain responsible for business rules and validation.

Atomic processing requirement

A DeliveryJob must not be processed by two workers or requests simultaneously.

The repository must use an atomic MongoDB operation to claim an eligible pending job.

For example, the status may transition from `PENDING` to `PROCESSING` only when the stored status is still `PENDING`.

If the job has already been claimed, another processing attempt must not send the same message.

This is particularly important when the application is running inside Docker or multiple processing requests occur concurrently.

5.3 Delivery Queue Creation

Existing endpoint:

```
POST /api/v1/deliveries
```

Delivery creation must require JWT authentication.

The workflow must:

1. Authenticate the requesting user.
2. Verify the user's role and Campaign permission.
3. Retrieve the referenced Campaign.
4. Validate the Campaign's Advisor ownership.
5. Verify that the Campaign is `ACTIVE`.
6. Verify that the delivery channel matches the Campaign channel.
7. Validate the recipient or publishing destination.
8. Verify marketing consent where applicable.
9. Verify audience eligibility.
10. Validate the requested delivery content.
11. Validate scheduling information.
12. Create a persistent DeliveryJob in MongoDB.
13. Return the queued delivery information.

Client-supplied `advisorId` values must not be trusted as proof of authorization.

The correct AdvisorProfile ID must be derived or verified against the referenced Campaign.

Marketing Specialists must have the required accepted advisor relationship and Campaign assignment.

The Delivery module must not allow a user to queue jobs for arbitrary Campaigns by submitting another Campaign ID.

5.4 Delivery Status and Lifecycle Management

The existing DeliveryJob lifecycle must be preserved and extended where necessary for reliable queue processing.

Recommended statuses:

- `PENDING`
- `PROCESSING`
- `SENT`
- `FAILED`

The following transitions should be enforced:

Current Status	Allowed Next Status
PENDING	PROCESSING
PROCESSING	SENT, FAILED
SENT	No further processing
FAILED	No automatic reprocessing

Any retry support must use an explicitly defined retry workflow.

A delivery already marked `SENT` must never be sent again through an ordinary processing request.

A delivery already being processed must not be claimed again.

The system must record meaningful timestamps and results for each processing attempt.

A delivery may be marked SENT only after the selected provider reports successful acceptance of the operation. Provider acceptance must not be confused with confirmed recipient inbox delivery.

For the mock provider, a simulated successful result must be clearly identifiable as a mock outcome.

5.5 Delivery Processing and Scheduling

Existing endpoint:

`POST /api/v1/deliveries/:id/process`

This endpoint must be updated to use MongoDB and shared JWT authorization.

The processing workflow must:

1. Authenticate the requesting user.
2. Verify the user's role and Campaign access.
3. Retrieve the DeliveryJob.
4. Confirm that the job is eligible for processing.
5. Verify that the scheduled execution time has arrived.
6. Revalidate the Campaign status and ownership.
7. Revalidate recipient consent and audience eligibility.
8. Atomically claim the job.
9. Execute the selected provider operation.
10. Persist the result.
11. Return the updated DeliveryJob.

Scheduling requirements

A future `scheduledAt` value must prevent early execution.

The module must support retrieving jobs whose scheduled time has arrived.

A lightweight queue processor may be implemented within the Delivery module.

The processing mechanism must not depend on Member 5's Automation scheduler.

Automation may request a future delivery, but the Delivery module remains responsible for deciding when its own queued jobs are eligible for processing.

A controlled polling worker or explicit processing mechanism may be used.

The worker must use the shared MongoDB connection and must avoid creating multiple uncontrolled processing loops.

Processing reliability

The implementation must prevent simultaneous claims and duplicate processing attempts.

For real external providers, a network failure may occur after a provider has accepted a message but before the database records success.

Such uncertain outcomes must not be automatically retried without appropriate safeguards.

Idempotency tracking, provider message identifiers, and explicit retry policies should be used where supported.

5.6 Gmail SMTP Integration

The Delivery module must implement a controlled email provider using Gmail SMTP and Nodemailer.

The integration must support authorized email delivery to consenting recipients.

Provider requirements

The email provider must:

- Use credentials from environment variables.
- Authenticate using a supported Gmail SMTP authentication method.
- Never hardcode credentials.
- Validate recipient email addresses.
- Use the Campaign's permitted content.
- Return structured success or failure information.
- Record provider message identifiers when available.
- Handle authentication and connection errors safely.

Example environment variable names:

```
None
GMAIL_USER=
GMAIL_APP_PASSWORD=
```

The actual values must not be committed to GitHub.

Email safety requirements

- Emails must not be sent without the necessary marketing consent.
- Unsubscribed Leads must not receive marketing emails.
- Recipient email addresses must match the referenced Lead.
- Campaign audience eligibility must be verified.
- Email sending must be rate-limited.
- Sending must be restricted to authorized Campaigns.
- Unsubscribe requirements must be respected.
- Provider credentials must never appear in API responses or logs.

For testing, only consenting test recipients should be used.

The implementation must not attempt uncontrolled bulk marketing delivery.

Gmail account restrictions and sending limits must be respected.

Gmail SMTP does not necessarily confirm whether a message reached the recipient's inbox. Delivery status terminology must accurately reflect the available provider result.

5.7 Facebook Page Publishing Integration

The Delivery module must support text-based publishing to an authorized Facebook Page through the Meta Graph API when the required access and permissions are available.

Only Page publishing operations supported by the authorized application's permissions should be attempted.

Requirements

The implementation should:

- Use a configured Facebook Page ID.
- Use a valid authorized Page access token.
- Publish permitted text content.
- Validate Campaign channel compatibility.
- Record successful provider responses.
- Record failed publishing attempts.
- Prevent unauthorized users from initiating Page publishing.

Example environment variable names:

```
None
META_ACCESS_TOKEN=
META_PAGE_ID=
```

The actual credentials must remain private.

Permission limitations

Facebook Page publishing may require Meta application permissions, Page access, and approval.

If the necessary permissions are unavailable, the real provider must not be falsely represented as operational.

The existing mock provider must remain available so the application can demonstrate the publishing workflow without requiring live Meta integration.

A mock publishing result must be identified as simulated.

The Delivery module must not attempt to bypass provider permission requirements.

5.8 Provider Abstraction and Mock Provider

The existing Phase I mock provider must be retained.

A consistent provider interface should support:

- Mock email delivery.
- Gmail SMTP email delivery.
- Mock Facebook publishing.
- Authorized Meta Page publishing.

Provider selection must be controlled through secure backend configuration.

For example, a development environment may use mock providers while an authorized environment may use real integrations.

Provider-specific logic must remain separate from business validation and queue management.

The Delivery service should perform authorization and eligibility checks before invoking a provider.

The selected provider should return a consistent result structure containing:

- Success status.
- Provider identifier.
- Provider message or publishing ID.
- Error information when applicable.

Provider errors must be handled without exposing credentials.

5.9 Rate Limiting and Controlled Delivery

The Delivery module must implement controls to prevent excessive or repeated marketing delivery.

Required functionality includes:

- Limiting delivery processing frequency.
- Avoiding uncontrolled concurrent sends.
- Respecting provider sending limits.
- Preventing duplicate processing.
- Handling temporary provider failures.
- Recording failed attempts.
- Supporting safe scheduling.

Rate limiting must apply to actual provider operations, not merely to HTTP request frequency.

The implementation should use a configurable limit appropriate for demonstration and testing.

The selected rate-limiting mechanism must work with the Dockerized environment and should not depend on an additional infrastructure service unless explicitly coordinated.

The system must prioritize correctness and safety over high-volume message throughput.

5.10 Recipient Consent and Eligibility Enforcement

Recipient consent is one of the most important requirements of the Delivery module.

Before an email job is queued or processed, the backend must verify:

1. The referenced Lead exists.
2. The Lead belongs to the Campaign's Advisor.
3. The Lead has explicitly consented to marketing.
4. The Lead has not unsubscribed.
5. The recipient email matches the Lead's registered email.
6. The Lead belongs to the Campaign's eligible audience.
7. The Campaign remains eligible for delivery.

Consent must be checked both when the job is created and immediately before processing.

For example, if a Lead withdraws consent after a job was queued but before it is processed, the message must not be sent.

The Lead Management module owns the consent records.

The Delivery module is responsible for enforcing consent before sending.

The Campaigns module owns audience selection.

Member 4 must use the agreed lookup contracts rather than duplicating the entire Leads or Campaigns implementation.

5.11 Delivery History and Tracking

Existing endpoint:

```
GET /api/v1/deliveries
```

Delivery history must be retrieved from MongoDB.

Supported filters should include relevant fields such as:

- Advisor.

- Campaign.
- Channel.
- Delivery status.
- Date or scheduling range.

The implementation must:

- Authenticate the requesting user.
- Restrict results to authorized Campaigns.
- Prevent cross-advisor data exposure.
- Support reasonable pagination.
- Return stable sorting.
- Record successful and failed outcomes.
- Preserve processing timestamps.
- Include relevant provider result metadata.

Marketing Specialists must see only delivery history associated with Campaigns they are authorized to access.

An Advisor may view delivery history for their own Campaigns.

Sensitive provider credentials must never be included in history responses.

5.12 Secure Data Handling

The Delivery module must implement:

- JWT authentication.
- Role-based authorization.
- Advisor ownership verification.
- Campaign assignment verification.
- Request validation.
- MongoDB ObjectId validation.
- Protection against mass assignment.
- Recipient consent enforcement.
- Secure provider credential handling.
- Safe logging.
- Controlled external provider error responses.
- Duplicate processing prevention.
- Atomic database updates.
- Appropriate HTTP status codes.

The shared authentication middleware must be reused.

Member 4 must not implement a separate JWT system or modify the Identity role definitions.

Fields such as `advisorId`, `deliveryStatus`, `sentAt`, and provider result metadata must not be freely modifiable through ordinary client requests.

Delivery lifecycle changes must occur through controlled service operations.

---

6. Internal Contracts for Other Modules

The Delivery module must integrate with Identity, Leads, Campaigns, and Automation through agreed backend contracts.

These dependencies must be mockable during independent development.

6.1 Identity Integration

Member 1 provides:

- JWT authentication.
- Role validation.
- AdvisorProfile lookup.
- Accepted advisor-specialist relationship lookup.

Member 4 must use these contracts to authorize Delivery operations.

Member 4 must not implement or modify the invitation system.

6.2 Leads Integration

Member 2 provides:

- Lead existence lookup.
- Advisor ownership information.
- Recipient email.
- Marketing consent.
- Unsubscribe status.

- Lead lifecycle information where required.

Member 4 uses this data to verify recipient eligibility.

The Lead Management module must not be modified to contain Delivery-specific processing logic.

6.3 Campaigns Integration

Member 3 provides:

- Campaign existence lookup.
- Campaign ownership.
- Campaign channel.
- Campaign status.
- Eligible audience lookup.
- Specialist Campaign assignment verification.

Member 4 must verify that only an `ACTIVE` Campaign can queue or process deliveries.

The selected Delivery channel must match the Campaign channel.

Member 4 must not modify Campaign assignment records or Campaign lifecycle rules.

6.4 Automation Integration

Member 5 may request delivery actions through the agreed Delivery service contract.

For example, an Automation rule may schedule a follow-up email after a supported business event.

The Automation module may submit a validated request to queue a delivery.

The Delivery module remains responsible for:

- Recipient eligibility.
- Consent verification.
- Campaign status checks.
- Provider selection.
- Scheduling.
- Rate limiting.
- Actual delivery processing.

- Delivery result persistence.

Automation must not bypass Delivery validation or directly send emails.

Delivery events

The Delivery module may expose events such as:

- `DELIVERY_QUEUED`
- `DELIVERY_SENT`
- `DELIVERY_FAILED`

These event names must match the agreed Automation event contract before integration.

The events should include relevant identifiers and safe metadata.

Member 5 remains responsible for persisting ActivityEvents and processing automation triggers.

The Delivery module must remain independently testable when Automation is unavailable.

---

7. API Compatibility

The following Phase I endpoints must remain functional:

Method	Endpoint	Purpose
POST	`/api/v1/deliveries`	Queue DeliveryJob
GET	`/api/v1/deliveries`	Retrieve delivery history
POST	`/api/v1/deliveries/:id/process`	Process queued delivery

These endpoints must be updated to use persistent MongoDB storage and shared authentication.

The existing route paths and response formats should be preserved wherever practical.

The processing endpoint must enforce the delivery lifecycle and authorization rules.

Additional endpoints may be introduced when necessary for safe delivery management, but shared route changes must be coordinated with Member 1.

No frontend implementation is required for Phase II.

All Delivery functionality must be testable through Postman and automated tests.

---

8. Testing Requirements

The Delivery module must be tested using Postman and appropriate automated tests.

Tests must use MongoDB persistence and controlled sample data.

Real email and Facebook operations must not be required for automated testing.

The mock provider must be used for deterministic delivery tests.

At minimum, the following scenarios must be verified.

8.1 Database Persistence

- DeliveryJob creation stores records in MongoDB.
- DeliveryJobs can be retrieved by ID.
- Delivery history is retrieved correctly.
- Delivery status updates persist.
- Data remains available after backend restart.
- Invalid database records are rejected.
- Required indexes and schema validation are applied.

8.2 Authentication and Authorization

- Missing JWT is rejected.

- Invalid JWT is rejected.
- Expired JWT is rejected.
- Advisor can queue deliveries for their own Campaign.
- Advisor cannot queue deliveries for another Advisor's Campaign.
- Assigned Marketing Specialist can perform permitted Delivery operations.
- Unassigned Marketing Specialist is rejected.
- Revoked advisor-specialist relationships invalidate access.
- Unauthorized users cannot retrieve private delivery history.
- Client-supplied `advisorId` cannot override authenticated ownership.

8.3 Campaign Validation

- Missing Campaign is rejected.
- Campaign belonging to another Advisor is rejected.
- Campaign with incorrect channel is rejected.
- Non-active Campaign is rejected.
- Valid active Campaign is accepted.
- Invalid Campaign references are rejected.

8.4 Recipient Consent

- Email delivery to a consenting Lead succeeds through the mock provider.
- Email delivery without consent is rejected.
- Email delivery to an unsubscribed Lead is rejected.
- Email delivery with an incorrect recipient address is rejected.
- Email delivery to a Lead outside the Campaign audience is rejected.
- Consent withdrawal after queue creation prevents processing.
- A Lead belonging to another Advisor is rejected.

8.5 Queue Processing

- Valid pending jobs can be processed.
- Future scheduled jobs cannot be processed early.
- Concurrent processing attempts do not send the same job twice.
- A processed job cannot be processed again.
- Successful provider responses are recorded.
- Failed provider responses are recorded.
- Processing timestamps are stored.
- Atomic claim behavior prevents duplicate processing.

- Retry behavior follows the documented policy.

8.6 Provider Integration

- Mock email processing works.
- Mock Facebook publishing works.
- Provider selection follows backend configuration.
- Missing real provider credentials are handled safely.
- Provider failures produce controlled error information.
- Sensitive credentials are never returned or logged.
- Real provider integrations are tested only with authorized test accounts and permissions.

8.7 Delivery History

- Advisor filtering works.
- Campaign filtering works.
- Status filtering works.
- Channel filtering works.
- Pagination returns correct records.
- Unrelated Advisors cannot access another Advisor's history.
- Specialists see only authorized Campaign deliveries.
- Sensitive provider information is excluded.

8.8 Integration Contract Tests

- DeliveryJobs reference valid Campaigns.
- Email DeliveryJobs reference valid Leads.
- DeliveryJob ownership matches Campaign ownership.
- Campaign access restrictions are respected.
- Lead consent and unsubscribe status are respected.
- Automation can use the agreed queueing service contract.
- Delivery events follow the shared event contract.
- Delivery operations remain functional when external services are mocked.

---

9. Documentation and GitHub Requirements

Member 4 must provide module-specific documentation explaining:

- Delivery module responsibilities.
- MongoDB DeliveryJob schema.
- Field definitions and validation rules.
- AdvisorProfile, Campaign, and Lead relationships.
- Supported delivery channels.
- Delivery lifecycle statuses.
- Queue creation and processing.
- Scheduling behavior.
- Atomic processing and duplicate prevention.
- Gmail SMTP integration requirements.
- Facebook Page publishing requirements and limitations.
- Mock provider usage.
- Rate limiting.
- Marketing consent and unsubscribe enforcement.
- JWT authentication and role-based authorization.
- Specialist Campaign permissions.
- API endpoint usage.
- Delivery history and tracking.
- Internal integration contracts.
- Postman test scenarios.
- Any necessary changes from Phase I.

The documentation must explicitly state:

`advisorId` references `AdvisorProfile._id`, not `User._id`.

`leadId` references `Lead._id`.

`campaignId` references `Campaign._id`.

Marketing Specialist authorization requires an accepted advisor relationship and a valid Campaign assignment.

Provider secrets must never be committed to GitHub.

Any new required environment variable names must be documented and coordinated with Member 1 for inclusion in the shared `.env.example`.

Development must occur in:

`feature/delivery-p2-m4`

Changes must be committed with meaningful commit messages.

A pull request must be submitted targeting `main`.

The pull request must include:

- Completed functionality summary.
- Schema and repository changes.
- Authentication and authorization integration.
- Provider implementation details.
- Tests performed.
- Relevant setup instructions.
- External integration limitations, if applicable.

The module must remain independently testable before merging.

---

10. Completion Criteria

Member 4's Phase II responsibilities will be considered complete when:

- DeliveryJob is integrated with MongoDB using Mongoose.
- In-memory delivery storage has been replaced with persistent database operations.
- Delivery records remain available after backend restart.
- DeliveryJobs reference valid Campaigns and applicable Leads.
- `advisorId` consistently means `AdvisorProfile._id`.
- Required fields, defaults, indexes, and schema validation are implemented.
- Existing Delivery endpoints remain functional.
- Delivery endpoints require JWT authentication.
- Advisor ownership is enforced by the backend.
- Marketing Specialists can access only permitted deliveries for assigned Campaigns.

- Revoked advisor-specialist relationships invalidate Delivery access.
- Only eligible active Campaigns can queue or process deliveries.
- Delivery channel compatibility is validated.
- Recipient marketing consent is checked before queueing and processing.
- Unsubscribed Leads cannot receive marketing emails.
- Recipient email and Campaign audience eligibility are validated.
- Delivery lifecycle transitions are enforced.
- DeliveryJobs are claimed atomically for processing.
- Duplicate or concurrent processing is prevented.
- Future scheduled jobs cannot be processed early.
- Controlled queue processing and rate limiting are implemented.
- Gmail SMTP provider integration is implemented with secure credential handling.
- Authorized Facebook Page publishing is supported where permissions are available.
- Mock email and Facebook providers remain functional.
- Provider outcomes and failures are recorded persistently.
- Delivery history supports filtering and authorized access.
- Automation can use the agreed Delivery queueing contract.
- No unrelated module's business logic has been implemented unnecessarily.
- Postman and automated tests cover successful and unsuccessful operations.
- Module documentation is complete.
- Changes are committed and submitted through a pull request.

Expected outcome: The Delivery module will operate using persistent MongoDB storage, with secure Campaign-based authorization, consent-aware recipient validation, reliable queue processing, delivery tracking, and controlled marketing provider integrations. Financial Advisors will manage deliveries for their own Campaigns, while authorized Marketing Specialists will perform only permitted operations for assigned Campaigns. The module will remain independently testable through mock providers and will support Automation through stable backend contracts without depending on another member's implementation.